Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Design and Analysis of Algorithm (CSDS103)
Lab Assignment: Comprehensive Algorithmic Analysis
Lab Assignment 10: Backtracking Algorithms

1 Introduction

Backtracking is a systematic algorithmic technique for solving constraint satisfaction problems (CSPs) by incrementally building candidates to the solutions and abandoning a candidate ("pruning" the branch) as soon as it is determined that the candidate cannot possibly lead to a valid full solution.

This laboratory assignment implements and evaluates backtracking algorithms across three classic problems with advanced constraints:

1. **Experiment 1**: Constrained 8-Queens Problem (with forbidden cells and pre-placed queens)
2. **Experiment 2**: Constrained Sum of Subsets Problem (with cardinality bounds, mandatory/excluded items, and branch pruning diagnostics)
3. **Experiment 3**: List-Constrained Graph Coloring Problem (with vertex color lists, pre-colored vertices, and search statistics)

All implementations are self-contained in Python and directly correspond to the accompanying interactive Jupyter Notebook `Backtracking_Lab_Assignment.ipynb`.

2 Experiment 1: Constrained 8-Queens Problem (Question 1.1)

2.1 Problem Statement & Requirements

Design a backtracking algorithm to solve the 8-Queens problem on an 8×8 chessboard such that no two queens share the same row, column, or diagonal. The algorithm extends the standard problem by supporting:

- **Forbidden cells**: A set of board coordinates (r, c) where queens cannot be placed.
- **Pre-placed queens**: Fixed queen positions (r, c) that must remain intact.
- **Input validation & Contradiction rejection**: Contradictory inputs (e.g., pre-placed queens on forbidden cells or attacking each other) must be rejected immediately before search.
- **Immediate branch pruning**: The branch must be pruned as soon as a queen placement conflicts with any previous queen or forbidden cell.

- **Symmetry reduction**: Support reducing symmetric branches when the constraints permit.
- **Solution enumeration**: Print all valid board arrangements and report the total count.

2.2 Algorithmic Formulation

A board state is represented as a 1D array `board[0..7]`, where `board[r] = c` indicates that the queen in row r is placed in column c. The backtracking search proceeds row-by-row from $r = 0$ to 7:

- If row r has a pre-placed queen, we verify its safety with previously placed rows; if safe, we recurse to row $r + 1$, else backtrack.
- If row r is unconstrained, we test column candidates $c \in \{0, \ldots, 7\}$ using the bounding function `is_safe(r, c)`. A placement is safe if $(r, c) \notin$ forbidden, $c \neq \text{board}[i]$, and $|r - i| \neq |c - \text{board}[i]|$ for all $0 \leq i < r$.

2.3 Python Source Code Implementation

The complete implementation of the solver function and board display helper is shown below:

```
def solve_constrained_8_queens(forbidden=None, pre_placed=None, symmetry_reduction=
    False):
    """
    Backtracking solver for the Constrained 8-Queens Problem.

    Parameters:
    - forbidden: set of (row, col) tuples where queens cannot be placed.
    - pre_placed: dict {row: col} representing fixed queen positions.
    - symmetry_reduction: bool, if True and constraints allow, reduces symmetric
    branches.

    Returns:
    - solutions: list of board configurations (each is a list where index=row, value=
    col).
    """
    if forbidden is None:
        forbidden = set()
    if pre_placed is None:
        pre_placed = {}

    # Requirement 4: Reject contradictory inputs
    # Check if any pre-placed queen occupies a forbidden cell
    for r, c in pre_placed.items():
        if (r, c) in forbidden:
            raise ValueError(f"Contradictory input: Pre-placed queen at ({r}, {c}) is
    in a forbidden cell.")

    # Check if any two pre-placed queens attack each other
    placed_list = list(pre_placed.items())
    for i in range(len(placed_list)):
        for j in range(i + 1, len(placed_list)):
            r1, c1 = placed_list[i]
```

```
            r2, c2 = placed_list[j]
            if r1 == r2 or c1 == c2 or abs(r1 - r2) == abs(c1 - c2):
                raise ValueError(f"Contradictory input: Pre-placed queens ({r1}, {c1}) and ({r2}, {c2}) attack each other.")

    solutions = []
    board = [-1] * 8  # board[r] represents the column of the queen in row r

    def is_safe(r, c):
        # Forbidden position check
        if (r, c) in forbidden:
            return False
        # Requirement 5: Prune immediately when attacking a previously placed queen
        for prev_r in range(r):
            prev_c = board[prev_r]
            if prev_c == c or abs(prev_r - r) == abs(prev_c - c):
                return False
        return True

    def backtrack(r):
        # Base case: All 8 queens placed successfully
        if r == 8:
            solutions.append(list(board))
            return

        # If current row has a pre-placed queen, respect it
        if r in pre_placed:
            c = pre_placed[r]
            if is_safe(r, c):
                board[r] = c
                backtrack(r + 1)
                board[r] = -1
            return

        col_range = range(8)
        # Requirement 6: Symmetry reduction (reflect horizontally across middle columns if unconstrained)
        if r == 0 and symmetry_reduction and not pre_placed and not forbidden:
            col_range = range(4)  # Explore only first 4 columns of row 0

        for c in col_range:
            if is_safe(r, c):
                board[r] = c
                backtrack(r + 1)
                board[r] = -1

    backtrack(0)
    return solutions


def print_board(solution):
    """Prints an 8x8 chessboard arrangement cleanly."""
    for r in range(8):
        row_str = " ".join("Q" if solution[r] == c else "." for c in range(8))
        print(f"Row {r}: {row_str}")
    print()
```

2.4 Demonstration Test Code

The demonstration code evaluates unconstrained 8-Queens, constrained boards with pre-placed queens and forbidden cells, and verifies rejection of contradictory input:

```
print("=== Experiment 1: Constrained 8-Queens Demonstrations ===")

# Test 1: Standard 8-Queens (no constraints)
all_solutions = solve_constrained_8_queens()
print(f"1. Standard 8-Queens total valid arrangements: {len(all_solutions)}")

# Test 2: Pre-placed Queen at (0, 0) and forbidden cells
forbidden = {(1, 2), (2, 4), (3, 1)}
pre_placed = {0: 0}  # Queen at row 0, col 0
constrained_solutions = solve_constrained_8_queens(forbidden=forbidden, pre_placed=
    pre_placed)
print(f"2. Constrained arrangements (pre-placed (0,0), forbidden={(1,2),(2,4),(3,1)}):
     {len(constrained_solutions)}")
print("\nFirst valid arrangement:")
print_board(constrained_solutions[0])

# Test 3: Contradictory Input Handling
print("3. Testing Contradictory Input Detection:")
try:
    # Two pre-placed queens in attack position: (0, 0) and (1, 1) on same diagonal
    solve_constrained_8_queens(pre_placed={0: 0, 1: 1})
except ValueError as e:
    print("  Successfully rejected:", e)

try:
    # Pre-placed queen on a forbidden cell
    solve_constrained_8_queens(forbidden={(0, 3)}, pre_placed={0: 3})
except ValueError as e:
    print("  Successfully rejected:", e)
```

2.5 Execution Results & Output Logs

The execution outputs obtained from the interactive notebook are as follows:

```
=== Experiment 1: Constrained 8-Queens Demonstrations ===
1. Standard 8-Queens total valid arrangements: 92
2. Constrained arrangements (pre-placed (0,0), forbidden=((1, 2), (2, 4), (3, 1))): 3

First valid arrangement:
Row 0: Q . . . . . . .
Row 1: . . . . Q . . .
Row 2: . . . . . . . Q
Row 3: . . . . . Q . .
Row 4: . . Q . . . . .
Row 5: . . . . . . Q .
Row 6: . Q . . . . . .
Row 7: . . . Q . . . .

3. Testing Contradictory Input Detection:
  Successfully rejected: Contradictory input: Pre-placed queens (0, 0) and (1, 1)
    attack each other.
```

```
Successfully rejected: Contradictory input: Pre-placed queen at (0, 3) is in a
  forbidden cell.
```

2.6 Analysis

- **Classical 8-Queens**: The algorithm discovers all 92 valid solutions on the unconstrained 8×8 board.
- **Constrained Search**: Placing a fixed queen at $(0, 0)$ and restricting coordinates $\{(1, 2), (2, 4), (3, 1)\}$ prunes the search space from 92 to exactly 3 valid arrangements, confirming immediate branch pruning.
- **Contradiction Rejection**: The algorithm rejects diagonal conflicts between pre-placed queens (e.g., $(0, 0)$ and $(1, 1)$) and forbidden cell overlaps before any recursive exploration.

3 Experiment 2: Constrained Sum of Subsets Problem (Question 2.1)

3.1 Problem Statement & Requirements

Given a set of n positive integers S and a target sum T, find all subsets whose elements sum exactly to T. The algorithm extends classical subset sum backtracking with the following constraints:

- **Cardinality constraints**: Minimum and maximum subset size bounds $[k_{\min}, k_{\max}]$.
- **Mandatory elements**: Specified element values that must appear in every valid subset.
- **Excluded elements**: Specified element values that cannot appear in any valid subset.
- **Positional index tracking**: Duplicate numerical values are distinguished by their indices.
- **Pruning criteria**:
  1. Prune branch when current sum exceeds T.
  2. Prune branch when maximum possible sum of remaining eligible elements cannot reach T.
  3. Prune branch when cardinality bounds $[k_{\min}, k_{\max}]$ cannot be satisfied.
- **Benchmark input**: $S = \{3, 5, 6, 7, 9, 11, 12\}$, $T = 21$, $k_{\min} = 2$, $k_{\max} = 4$, Mandatory $= \{3\}$, Excluded $= \{7\}$ Every valid subset must be reported, and every pruned branch must be logged with its reason.

3.2 Algorithmic Formulation

The search tree is a binary decision tree where each node corresponds to the decision of including or excluding item $i \in S$. The state is given by $(\text{idx}, \text{current_indices}, \text{current_sum})$. Before exploring subtrees:

1. If current_sum $= T$, we verify $k_{\min} \leq |\text{current_indices}| \leq k_{\max}$ and check mandatory containment.
2. If current_sum $> T$, prune (monotonicity of positive integers).
3. If $|\text{current_indices}| \geq k_{\max}$, prune further expansion.
4. If $|\text{current_indices}| + \text{remaining_elements} < k_{\min}$, prune (cardinality underflow).
5. If current_sum $+ \sum_{j=\text{idx}}^{n-1} S[j] < T$, prune (insufficient remaining sum).
6. If prospective sum current_sum $+ S[\text{idx}] > T$, prune the inclusion branch.

3.3 Python Source Code Implementation

The complete implementation of the constrained sum of subsets solver is shown below:

```
def solve_constrained_sum_of_subsets(S, T, k_min, k_max, mandatory=None, excluded=None
    ):
    """
    Backtracking solver for the Constrained Sum of Subsets Problem.

    Parameters:
    - S: list of positive integers.
    - T: target sum.
    - k_min: minimum permitted subset size.
    - k_max: maximum permitted subset size.
    - mandatory: list of element values that must appear.
    - excluded: list of element values that cannot appear.

    Returns:
    - valid_subsets: list of valid subsets (each as a list of values).
    - pruning_logs: list of (current_subset, reason) explaining pruned branches.
    """
    if mandatory is None:
        mandatory = []
    if excluded is None:
        excluded = []

    mand_set = set(mandatory)
    excl_set = set(excluded)

    # Requirement 4: Reject invalid input
    if mand_set & excl_set:
        raise ValueError("Invalid input: Mandatory and excluded elements overlap.")
    if k_min > k_max or k_min < 0:
        raise ValueError("Invalid input: k_min cannot exceed k_max.")
    for m in mand_set:
        if m not in S:
```

```
        raise ValueError(f"Invalid input: Mandatory element {m} is not in input set S.")

    # Requirement 3: Distinguish duplicate values by tracking (index, value)
    items = [(i, val) for i, val in enumerate(S) if val not in excl_set]

    valid_subsets = []
    pruning_logs = []

    def backtrack(idx, current_indices, current_sum):
        current_vals = [S[i] for i in current_indices]

        # Check if target sum is met
        if current_sum == T:
            # Check cardinality constraint
            if k_min <= len(current_indices) <= k_max:
                # Check mandatory elements requirement
                if mand_set.issubset(set(current_vals)):
                    valid_subsets.append(list(current_vals))
            # Since all elements are positive, adding further elements will exceed T
            return

        if idx >= len(items):
            return

        # Pruning 1: Current sum exceeds T
        if current_sum > T:
            pruning_logs.append((current_vals, f"Current sum ({current_sum}) exceeds target ({T})"))
            return

        # Pruning 7a: Cardinality limit exceeded
        if len(current_indices) >= k_max:
            pruning_logs.append((current_vals, f"Subset size reached maximum permitted cardinality k_max={k_max}"))
            return

        # Pruning 7b: Cannot satisfy minimum cardinality k_min
        remaining_count = len(items) - idx
        if len(current_indices) + remaining_count < k_min:
            pruning_logs.append((current_vals, f"Remaining elements ({remaining_count}) cannot reach minimum cardinality k_min={k_min}"))
            return

        # Pruning 6: Maximum possible sum of remaining eligible elements cannot reach T
        remaining_sum = sum(val for _, val in items[idx:])
        if current_sum + remaining_sum < T:
            pruning_logs.append((current_vals, f"Max achievable sum ({current_sum + remaining_sum}) is less than target ({T})"))
            return

        item_idx, item_val = items[idx]

        # Branch 1: Include items[idx]
        # Pruning 5: Prune immediately if adding item_val exceeds T
        if current_sum + item_val <= T:
            backtrack(idx + 1, current_indices + [item_idx], current_sum + item_val)
```

```
        else:
            pruning_logs.append((current_vals + [item_val], f"Adding element {item_val} gives sum {current_sum + item_val} > {T}"))

        # Branch 2: Exclude items[idx]
        backtrack(idx + 1, current_indices, current_sum)

    backtrack(0, [], 0)
    return valid_subsets, pruning_logs
```

3.4 Benchmark Test Execution Code

The code executing the required test case $S = \{3, 5, 6, 7, 9, 11, 12\}, T = 21$ is given below:

```
print("=== Experiment 2: Benchmark Test Execution ===")

S = [3, 5, 6, 7, 9, 11, 12]
T = 21
k_min = 2
k_max = 4
mandatory = [3]
excluded = [7]

subsets, logs = solve_constrained_sum_of_subsets(S, T, k_min, k_max, mandatory, excluded)

print(f"Given Set S = {S}")
print(f"Target T = {T}")
print(f"Cardinality range: [{k_min}, {k_max}]")
print(f"Mandatory elements: {mandatory}")
print(f"Excluded elements: {excluded}")
print("-" * 50)

print(f"Total Valid Subsets Found: {len(subsets)}")
for idx, s in enumerate(subsets, 1):
    print(f"  Valid Subset {idx}: {s} (Sum = {sum(s)}, Size = {len(s)})")

print("\n" + "-" * 50)
print(f"Pruned Branches Report (Total Pruned: {len(logs)}):")
print("-" * 50)
for branch, reason in logs:
    branch_str = str(branch) if branch else "[]"
    print(f"Branch {branch_str.ljust(22)} -> {reason}")
```

3.5 Execution Results & Output Logs

The complete execution results, valid subsets, and detailed pruning logs are shown below:

```
=== Experiment 2: Benchmark Test Execution ===
Given Set S = [3, 5, 6, 7, 9, 11, 12]
Target T = 21
Cardinality range: [2, 4]
Mandatory elements: [3]
Excluded elements: [7]
```

```
------------------------------------------------
Total Valid Subsets Found: 1
  Valid Subset 1: [3, 6, 12] (Sum = 21, Size = 3)

------------------------------------------------
Pruned Branches Report (Total Pruned: 29):
------------------------------------------------
Branch [3, 5, 6, 9]           -> Adding element 9 gives sum 23 > 21
Branch [3, 5, 6, 11]          -> Adding element 11 gives sum 25 > 21
Branch [3, 5, 6, 12]          -> Adding element 12 gives sum 26 > 21
Branch [3, 5, 9, 11]          -> Adding element 11 gives sum 28 > 21
Branch [3, 5, 9, 12]          -> Adding element 12 gives sum 29 > 21
Branch [3, 5, 11, 12]         -> Adding element 12 gives sum 31 > 21
Branch [3, 5]                 -> Max achievable sum (20) is less than target (21)
Branch [3, 6, 9, 11]          -> Adding element 11 gives sum 29 > 21
Branch [3, 6, 9, 12]          -> Adding element 12 gives sum 30 > 21
Branch [3, 6, 11, 12]         -> Adding element 12 gives sum 32 > 21
Branch [3, 9, 11]             -> Adding element 11 gives sum 23 > 21
Branch [3, 9, 12]             -> Adding element 12 gives sum 24 > 21
Branch [3, 11, 12]            -> Adding element 12 gives sum 26 > 21
Branch [3]                    -> Max achievable sum (15) is less than target (21)
Branch [5, 6, 9, 11]          -> Adding element 11 gives sum 31 > 21
Branch [5, 6, 9, 12]          -> Adding element 12 gives sum 32 > 21
Branch [5, 6, 11]             -> Adding element 11 gives sum 22 > 21
Branch [5, 6, 12]             -> Adding element 12 gives sum 23 > 21
Branch [5, 9, 11]             -> Adding element 11 gives sum 25 > 21
Branch [5, 9, 12]             -> Adding element 12 gives sum 26 > 21
Branch [5, 11, 12]            -> Adding element 12 gives sum 28 > 21
Branch [5]                    -> Max achievable sum (17) is less than target (21)
Branch [6, 9, 11]             -> Adding element 11 gives sum 26 > 21
Branch [6, 9, 12]             -> Adding element 12 gives sum 27 > 21
Branch [6, 11, 12]            -> Adding element 12 gives sum 29 > 21
Branch [6]                    -> Max achievable sum (18) is less than target (21)
Branch [9, 11, 12]            -> Adding element 12 gives sum 32 > 21
Branch [11, 12]               -> Adding element 12 gives sum 23 > 21
Branch []                     -> Remaining elements (1) cannot reach minimum
    cardinality k_min=2
```

3.6 Pruning Diagnostics Analysis

On the test set $S = \{3, 5, 6, 7, 9, 11, 12\}$ with excluded element 7 and mandatory element 3:

- **Unique Valid Solution**: Exactly one subset meets all constraints:

$$\{3, 6, 12\} \quad (\text{Sum} = 21, \quad \text{Size} = 3 \in [2, 4], \quad 3 \in \text{subset}, \quad 7 \notin \text{subset})$$

- **Sum Exceeded Pruning**: Branches such as $[3, 5, 6, 9]$ were pruned because adding 9 produced sum $23 > 21$.
- **Max Achievable Sum Pruning**: Branch $[3, 5]$ was pruned because the maximum sum achievable with all remaining elements was only $20 < 21$.
- **Cardinality Underflow Pruning**: Branches with insufficient remaining elements to reach $k_{\min} = 2$ were pruned immediately.

4 Experiment 3: Graph Coloring Using Backtracking (Question 3.1)

4.1 Problem Statement & Requirements

Given an undirected graph $G = (V, E)$, assign colors to its vertices such that no two adjacent vertices share the same color. The algorithm extends standard m-coloring by supporting:

- **Maximum colors bound**: At most m available colors (integers $1, 2, \ldots, m$).
- **Allowed color lists**: Each vertex v has a restricted set allowed_colors$[v] \subseteq \{1, \ldots, m\}$.
- **Pre-colored vertices**: A set of fixed color assignments pre_colored$[v]$ that cannot be changed.
- **Adjacency conflict rejection**: Reject assignments where adjacent vertices receive the same color.
- **Constraint pruning**: Prune branches immediately when a color choice violates an adjacency or allowed list constraint.
- **Search statistics**: Report the total number of recursive calls made and branches pruned.

4.2 Algorithmic Formulation

Vertices are indexed $0, \ldots, |V| - 1$. The recursive function `backtrack(idx)` colors vertex $u = V[\text{idx}]$:

- If vertex u is pre-colored, verify that its pre-assigned color does not conflict with any colored neighbors in G. If safe, recurse to vertex idx + 1; otherwise, increment `pruned_branches` and backtrack.
- If vertex u is uncolored, iterate through colors $c \in$ allowed_colors$[u]$ where $c \leq m$. If c does not conflict with any adjacent neighbor (`is_safe(u, c)`), tentatively assign color$[u] = c$ and recurse; otherwise, increment `pruned_branches`.

4.3 Python Source Code Implementation

The complete implementation of the list-constrained graph coloring solver is shown below:

```
def solve_list_constrained_graph_coloring(adj, m, allowed_colors, pre_colored=None):
    """
    Backtracking solver for List-Constrained Graph Coloring.

    Parameters:
    - adj: dict {vertex: [neighbors]} representing an undirected graph.
    - m: maximum available colors (1 to m).
    - allowed_colors: dict {vertex: [allowed_color_integers]}.
    - pre_colored: dict {vertex: fixed_color}.

    Returns:
```

```
    - solutions: list of valid colorings (each is a dict {vertex: color}).
    - stats: dict containing 'recursive_calls' and 'pruned_branches'.
    """
    if pre_colored is None:
        pre_colored = {}

    vertices = list(adj.keys())
    n = len(vertices)

    # Requirement 4 & Input validation: Check pre-colored vertices
    for u, col in pre_colored.items():
        if col not in allowed_colors.get(u, []):
            raise ValueError(f"Contradictory input: Pre-colored vertex {u} has color {col} not in its allowed list.")
        if col > m or col < 1:
            raise ValueError(f"Contradictory input: Pre-colored vertex {u} has color {col} exceeding available colors {m}.")
        # Check pre-colored adjacency conflict
        for v in adj[u]:
            if v in pre_colored and pre_colored[v] == col:
                raise ValueError(f"Contradictory input: Adjacent pre-colored vertices {u} and {v} both have color {col}.")

    colors = {u: None for u in vertices}
    for u, col in pre_colored.items():
        colors[u] = col

    solutions = []
    stats = {"recursive_calls": 0, "pruned_branches": 0}

    def is_safe(u, col):
        # Requirement 3: Reject any assignment in which adjacent vertices share a color
        for v in adj[u]:
            if colors[v] == col:
                return False
        return True

    def backtrack(idx):
        stats["recursive_calls"] += 1

        # Base case: All vertices colored
        if idx == n:
            solutions.append(dict(colors))
            return

        u = vertices[idx]

        # If vertex is pre-colored, preserve its color and verify adjacency
        if u in pre_colored:
            if is_safe(u, pre_colored[u]):
                backtrack(idx + 1)
            else:
                stats["pruned_branches"] += 1
            return

        # Requirement 5: Try only colors from the vertex's allowed list up to m
        for col in allowed_colors.get(u, []):
```

```
            if col > m:
                continue
            if is_safe(u, col):
                colors[u] = col
                backtrack(idx + 1)
                colors[u] = None
            else:
                # Requirement 6: Prune branch as soon as color violates adjacency
                stats["pruned_branches"] += 1

    backtrack(0)
    return solutions, stats
```

4.4 Demonstration Test Code

The demonstration code evaluates a 5-vertex undirected graph with cycle and cross edges, specific vertex allowed lists, and a pre-colored vertex:

```
print("=== Experiment 3: List-Constrained Graph Coloring Demonstrations ===")

# Undirected graph G with 5 vertices (V0 to V4):
# Edges: (0-1), (0-2), (1-2), (1-3), (2-4), (3-4)
adj = {
    0: [1, 2],
    1: [0, 2, 3],
    2: [0, 1, 4],
    3: [1, 4],
    4: [2, 3]
}

m = 3  # Colors: 1, 2, 3

# Vertex-specific allowed color lists
allowed_colors = {
    0: [1, 2],
    1: [1, 2, 3],
    2: [2, 3],
    3: [1, 3],
    4: [1, 2, 3]
}

# Pre-colored vertex: Vertex 0 is fixed to color 1
pre_colored = {0: 1}

colorings, stats = solve_list_constrained_graph_coloring(adj, m, allowed_colors,
    pre_colored)

print(f"Graph vertices: {list(adj.keys())}")
print(f"Max colors (m): {m}")
print(f"Allowed colors per vertex: {allowed_colors}")
print(f"Pre-colored vertices: {pre_colored}")
print("-" * 50)
print(f"Valid Colorings Found: {len(colorings)}")
for i, col_map in enumerate(colorings, 1):
    print(f"  Coloring {i}: {col_map}")

```

```
print("-" * 50)
print("Search Statistics:")
print(f"  Total recursive calls: {stats['recursive_calls']}")
print(f"  Total branches pruned: {stats['pruned_branches']}")

# Test Contradictory Input: Adjacent vertices pre-colored with same color
print("\nTesting Contradictory Input Detection:")
try:
    solve_list_constrained_graph_coloring(adj, m, allowed_colors, pre_colored={0: 1, 1: 1})
except ValueError as e:
    print("  Successfully rejected:", e)
```

4.5 Execution Results & Output Logs

The execution outputs including valid colorings, search statistics, and contradictory input handling are shown below:

```
=== Experiment 3: List-Constrained Graph Coloring Demonstrations ===
Graph vertices: [0, 1, 2, 3, 4]
Max colors (m): 3
Allowed colors per vertex: {0: [1, 2], 1: [1, 2, 3], 2: [2, 3], 3: [1, 3], 4: [1, 2, 3]}
Pre-colored vertices: {0: 1}
--------------------------------------------------
Valid Colorings Found: 4
  Coloring 1: {0: 1, 1: 2, 2: 3, 3: 1, 4: 2}
  Coloring 2: {0: 1, 1: 2, 2: 3, 3: 3, 4: 1}
  Coloring 3: {0: 1, 1: 2, 2: 3, 3: 3, 4: 2}
  Coloring 4: {0: 1, 1: 3, 2: 2, 3: 1, 4: 3}
--------------------------------------------------
Search Statistics:
  Total recursive calls: 13
  Total branches pruned: 9

Testing Contradictory Input Detection:
  Successfully rejected: Contradictory input: Adjacent pre-colored vertices 0 and 1
    both have color 1.
```

4.6 Analysis

- **Constrained Search Efficiency**: For the 5-vertex test graph with $m = 3$, restricted color domains and pre-colored vertex $0 \mapsto 1$ yielded exactly 4 valid vertex colorings.
- **Search Statistics**: The search required 13 recursive calls and pruned 9 branches as soon as adjacency collisions or color list restrictions occurred.
- **Contradictory Pre-coloring Rejection**: Pre-coloring adjacent vertices 0 and 1 with identical color 1 was caught and rejected during input validation before recursion.

5 Conclusion

This lab demonstrated the effectiveness of backtracking in solving complex combinatorial search problems with domain and structural constraints. Across all three experiments (Constrained 8-Queens, Constrained Sum of Subsets, and List-Constrained Graph Coloring):

1. **Early Bounding**: Checking constraints prior to recursive descent dramatically shrinks the active search space compared to brute-force enumeration.
2. **Constraint Pruning**: Combining problem-specific bounds (e.g., maximum achievable sum, cardinality limits, diagonal collision checks) ensures early termination of infeasible branches.
3. **Input Validation**: Detecting contradictory constraints before initiating search preserves computational resources and avoids invalid states.